

# Prescription Trend AI — Dynamic AI-Agent Architecture

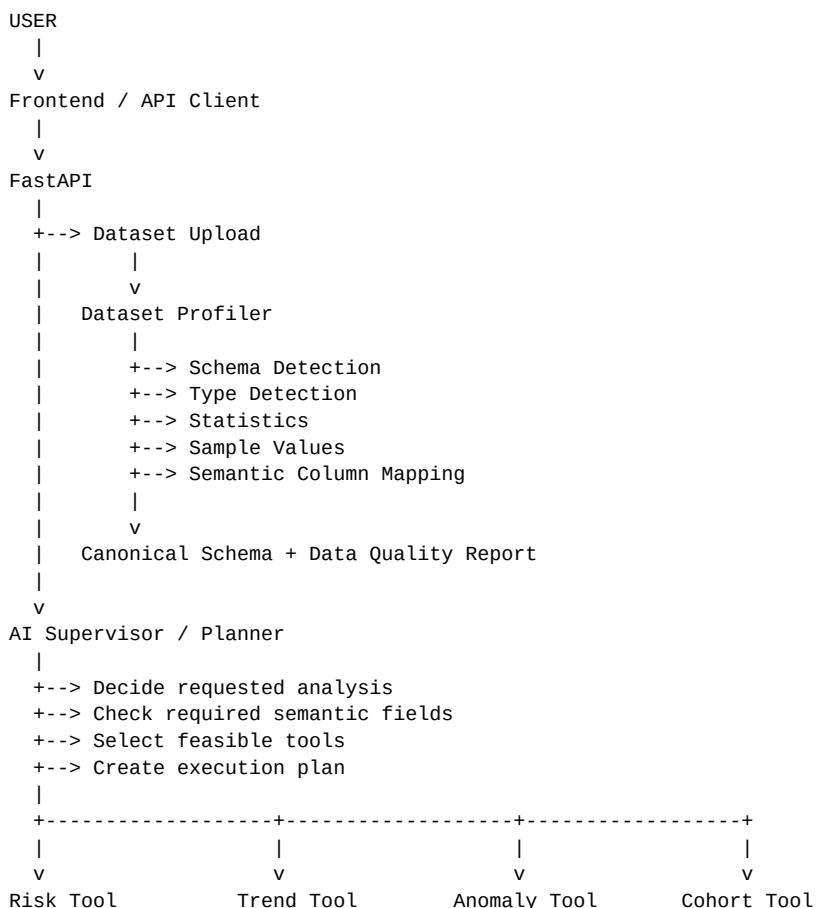
Complete implementation plan, architecture, decision flow, and implementation roadmap

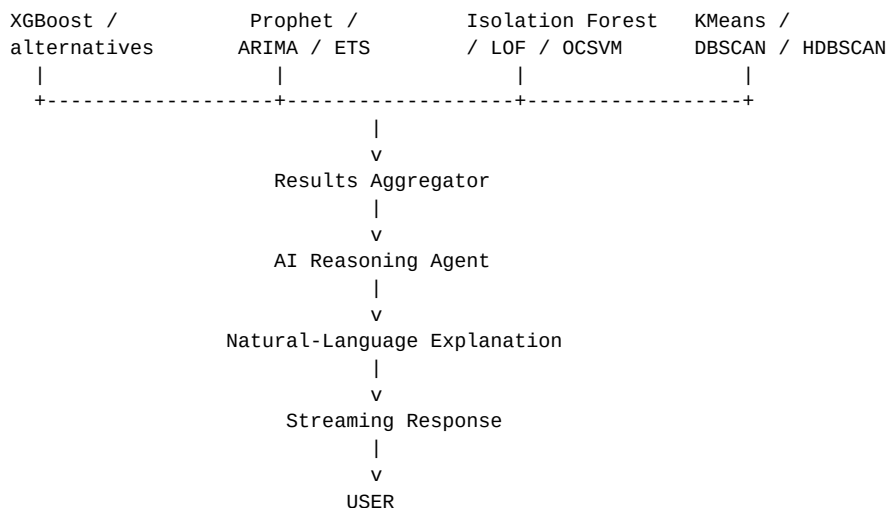
**Core goal:** Build a schema-independent Prescription Trend AI that accepts different datasets, understands their structure, dynamically selects appropriate analysis tools/models, runs independent tools concurrently where possible, and uses an AI agent to reason over measurable ML results and explain them.

## 1. Core Design Principles

- **No fixed dataset schema:** never assume columns such as drug\_name, patient\_id, date, age, etc. exist.
- **Canonical semantic schema:** map arbitrary source columns to meanings such as DRUG, DATE, PATIENT\_ID, DIAGNOSIS, AGE, GENDER, DOSAGE, REGION, QUANTITY, PRESCRIBER.
- **ML models are tools:** XGBoost, Prophet, Isolation Forest, KMeans, DBSCAN, HDBSCAN, etc. are specialized tools used by agents.
- **AI agents orchestrate:** an LLM-based agent decides what analysis is needed and which available tools can satisfy it.
- **Metrics decide model quality:** Python/ML evaluation code calculates scores. The LLM should not invent or guess which model is mathematically best.
- **Graceful degradation:** if required data is absent, the system must say the analysis is unavailable rather than fabricate data.
- **Parallel execution:** independent tools should run concurrently; dependent tools wait for required results.
- **Streaming:** stream the final AI explanation so users see progress/results without waiting for the entire response.

## 2. Complete High-Level Architecture





### 3. Step-by-Step Implementation Plan

#### Step 1 — Project foundation

Create a clean FastAPI backend with modules for dataset ingestion, profiling, semantic mapping, tools, model selection, agents, orchestration, result aggregation, and streaming.

#### Step 2 — Dataset ingestion

Accept CSV/XLSX or other supported tabular inputs. Store a safe working copy, calculate a dataset ID, and keep the raw dataset separate from transformed data.

#### Step 3 — Dataset Profiler

Automatically inspect column names, pandas dtypes, missing values, unique counts, cardinality, numeric ranges, date-likeness, categorical distributions, sample values, duplicate rows, and basic quality statistics.

#### Step 4 — Semantic schema mapping

Map source columns to canonical concepts. Use a layered approach: column-name rules/fuzzy matching first, data-type/content heuristics second, and LLM assistance only for ambiguous cases. Store mapping confidence and evidence.

#### Step 5 — Data-quality and capability report

Create a machine-readable report listing available semantic fields, missing fields, low-quality fields, and which analyses are possible. Never create fake values for missing fields.

#### Step 6 — Query understanding

Convert the user's question into an analysis intent such as trend, risk, anomaly, cohort, drug interaction/pattern, comparison, forecast, or combined analysis.

#### Step 7 — AI Supervisor / Planner

Given the query intent plus the dataset capability report, decide which tools are required, what inputs they need, and which calls can run independently.

#### Step 8 — Tool registry

Define every ML capability as a registered tool with a standard interface: name, purpose, required semantic fields, optional fields, input contract, output contract, evaluation method, and failure conditions.

#### Step 9 — Automatic model selection

For tasks where multiple algorithms are possible, run candidates on suitable transformed data, calculate objective metrics, check constraints, and select the best valid candidate. Record the selection evidence.

## **Step 10 — Clustering selection example**

For cohort analysis, compare KMeans, DBSCAN, HDBSCAN, and Agglomerative where applicable. Use Silhouette (higher is better), Davies-Bouldin (lower is better), Calinski-Harabasz (higher is better), cluster count, noise ratio, and stability/validity checks. DBSCAN can win when density/noise/shape characteristics make it more appropriate.

## **Step 11 — Risk tool**

Use XGBoost when a valid supervised target/label exists. If the dataset lacks a suitable target, do not pretend that supervised risk prediction is available; route to an appropriate unsupervised/statistical analysis or report the limitation.

## **Step 12 — Trend/forecast tool**

Require a usable date/time semantic field and a measurable series. Compare candidate forecasting methods where justified, validate on a holdout period, and record error metrics before selecting a model.

## **Step 13 — Anomaly tool**

Compare suitable anomaly methods according to data characteristics and validation strategy. Isolation Forest, LOF, or One-Class SVM are candidates; the final choice should be evidence-based.

## **Step 14 — Concurrent execution**

Run independent tools concurrently with Python async/concurrent execution. For example, Risk, Trend, Anomaly, and Cohort can run together if they do not depend on one another. Pattern analysis should wait if it requires outputs from earlier tools.

## **Step 15 — Result normalization**

Convert every tool output into a common result structure containing tool name, model used, inputs, metrics, findings, warnings, confidence/evidence, and machine-readable data.

## **Step 16 — AI reasoning/synthesis**

Give the AI reasoning agent the normalized results and the user's question. It explains relationships between findings, highlights uncertainty, and answers only from available evidence.

## **Step 17 — Streaming**

Stream the synthesized response from the backend to the frontend. Optionally stream structured progress events such as profiling complete, tool started, tool completed, and synthesis started.

## **Step 18 — Safety and correctness**

Validate uploads, avoid leaking sensitive raw data to the LLM, limit sample data sent for schema inference, log tool decisions, and ensure the system explicitly reports unsupported analyses.

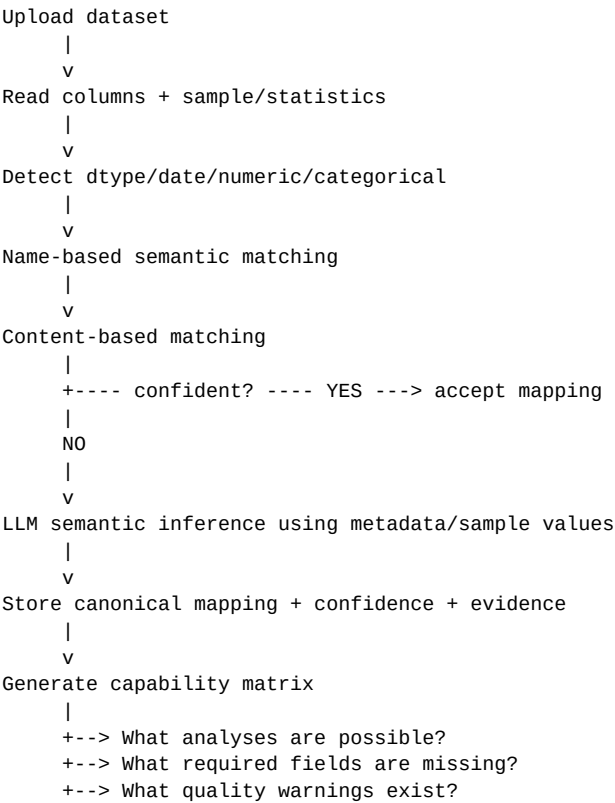
## **Step 19 — Testing**

Create test datasets with different column names, missing fields, noisy data, dates in different formats, categorical/numeric mixtures, and datasets where different algorithms win.

## **Step 20 — Deployment**

Deploy FastAPI and the model/LLM services separately or together as appropriate, use environment variables for secrets, add database/storage if needed, monitor latency, model selection, tool failures, and token generation.

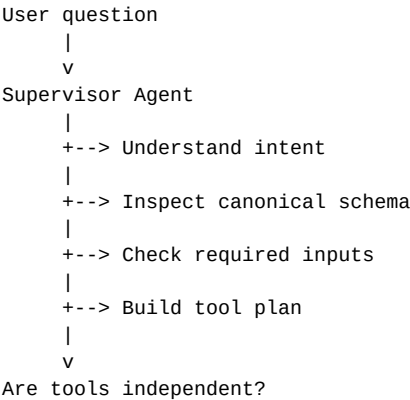
## 4. Dynamic Schema Flowchart

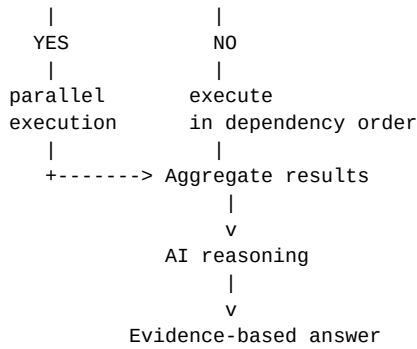


## 5. Canonical Schema Example

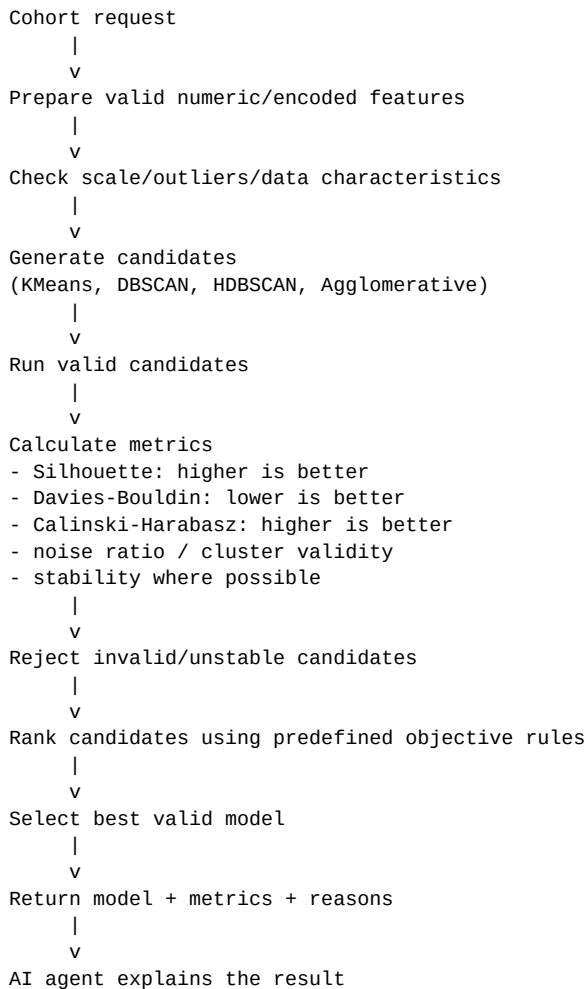
| Canonical field | Examples of source names                  | Typical detection evidence         |
|-----------------|-------------------------------------------|------------------------------------|
| DRUG            | drug_name, medicine, medication, compound | name similarity + drug-like values |
| DATE            | date, rx_date, prescribed_on              | date parsing                       |
| PATIENT_ID      | patient_id, patient_no, subject           | identifier/cardinality patterns    |
| DIAGNOSIS       | diagnosis, condition, disease             | categorical medical terms          |
| AGE             | age, patient_age                          | numeric range                      |
| GENDER          | gender, sex                               | small categorical domain           |
| DOSAGE          | dosage, dose                              | numeric/text dose patterns         |
| REGION          | region, location, area                    | categorical geographic values      |
| QUANTITY        | quantity, count, units                    | numeric distribution               |
| PRESCRIBER      | prescriber, physician, doctor             | name/identifier patterns           |

## 6. AI-Agent Decision Flow





## 7. Model Selection Flow — Example: Clustering



## 8. Tool Contract

Every tool should expose a standard contract:

Tool name  
 Purpose  
 Required semantic fields  
 Optional fields  
 Preprocessing requirements  
 Candidate models  
 Evaluation metrics  
 Input schema  
 Output schema  
 Failure conditions  
 Evidence / metrics  
 Execution time

Example:  
 CohortTool

requires: [AGE, QUANTITY] or other valid cohort features  
candidates: [KMeans, DBSCAN, HDBSCAN, Agglomerative]  
output: clusters + model + metrics + warnings

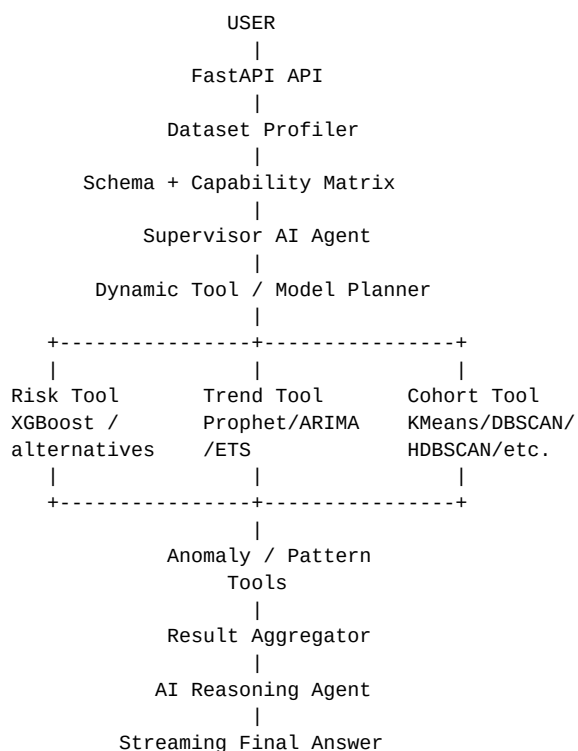
## 9. Result Normalization

```
{
  "tool": "cohort",
  "model": "DBSCAN",
  "status": "success",
  "inputs": ["AGE", "QUANTITY", "RISK_SCORE"],
  "metrics": {
    "silhouette": 0.67,
    "davies_bouldin": 0.45
  },
  "findings": [...],
  "warnings": [...],
  "evidence": [...]
}
```

## 10. Recommended Build Order

- Phase 1: FastAPI + upload + profiling.
- Phase 2: Canonical semantic mapping + capability matrix.
- Phase 3: Tool registry and standardized tool contracts.
- Phase 4: Individual ML tools with evaluation.
- Phase 5: Automatic model selection.
- Phase 6: Supervisor AI agent and planning.
- Phase 7: Concurrent execution and dependency handling.
- Phase 8: Result aggregation and evidence-based AI synthesis.
- Phase 9: Streaming progress/final response.
- Phase 10: Testing with many schemas and deployment.

## 11. Final Architecture to Target



## 12. Key Rule for the Project

**Do not build a fixed pipeline where every dataset always uses the same algorithm.** Build a dynamic system where the dataset is profiled first, capabilities are discovered, candidate tools/models are evaluated with objective metrics, the AI agent plans and orchestrates the available tools, and the final AI response explains the evidence. This is the core change from a fixed ML pipeline to a practical AI-agent architecture.